LAB 5_ Assignment: Implementation of Chapter-3

Course Name and instructor: GAI

Prof G C Nandi

Chapter 3 Summary: Looking Inside Large Language Models

Chapter 4 Text classification using pretrained language models.

What You Need to Know:

Think of LLMs as **super-powered autocomplete systems** that learned patterns from reading most of the internet. Here's what happens under the hood:

Key Components:

1. **Transformer Architecture** - The brain structure that processes words in parallel (not sequentially like older models)
2. **Tokenization** - Breaking text into pieces (tokens) that the model understands (not just words - sometimes parts of words!)
3. **Embeddings** - Turning tokens into number sequences (vectors) that capture meaning
4. **Attention Mechanism** - The magic that lets the model understand which words are important in relation to others
5. **Layers & Parameters** - Stacked processing steps (like a factory assembly line) with billions of adjustable knobs

How They Work:

- **Training Phase**: Models read vast text and learn to predict the next word (like a giant guessing game)

- **Inference Phase**: You give a prompt, and the model generates what comes next based on learned patterns
- **No "Understanding"**: Despite seeming intelligent, LLMs are pattern-matching statistical engines

Why This Matters:

- Knowing the architecture helps you understand model limitations
- Different models (Phi-3-mini-4k, GPT, BERT, T5) have different internal designs for different tasks
- The "black box" isn't completely black - we can peek inside with visualization tools

Coding Assignment:

Objective: Visualize how the Phi-3-mini-4k model processes text and understand its architecture.

```
# Coding Exercise - Exploring Phi-3-mini Architecture
# Make sure you have downloaded Phi-3-mini-4k-instruct
# Install: pip install transformers torch accelerate

from transformers import AutoTokenizer, AutoModelForCausalLM
import torch
import matplotlib.pyplot as plt
import numpy as np

# 1. Load the Phi-3-mini model and tokenizer (assuming downloaded locally)
model_path = "./phi-3-mini-4k-instruct"  # Update with your local path
tokenizer = AutoTokenizer.from_pretrained(model_path, trust_remote_code=True)
model = AutoModelForCausalLM.from_pretrained(
    model_path,
    torch_dtype=torch.float32,  # Use float16 or bfloat16 if GPU available
    trust_remote_code=True,
    device_map="auto"  # Automatically places layers on available devices
)

# 2. Explore model architecture
print("=== Phi-3-mini Architecture ===")
print(f"Model type: {type(model).__name__}")
print(f"Number of parameters: {sum(p.numel() for p in model.parameters()):,}")
```

```python
print(f"Model layers: {model.config.num_hidden_layers}")
print(f"Hidden size: {model.config.hidden_size}")
print(f"Attention heads: {model.config.num_attention_heads}")
print(f"Vocabulary size: {model.config.vocab_size}")
print(f"Max position embeddings: {model.config.max_position_embeddings}")

# 3. Tokenization analysis
text = "The quick brown fox jumps over the lazy dog"
tokens = tokenizer.encode(text)
decoded_tokens = [tokenizer.decode([token]) for token in tokens]

print(f"\n=== Tokenization Analysis ===")
print(f"Original text: '{text}'")
print(f"Token IDs: {tokens}")
print(f"Decoded tokens: {decoded_tokens}")
print(f"Number of tokens: {len(tokens)}")

# 4. Forward pass with attention capture
def get_attention_patterns(text, layer_idx=5):
    """Extract attention patterns from a specific layer"""
    inputs = tokenizer(text, return_tensors="pt")

    # Get model outputs with attention
    with torch.no_grad():
        outputs = model(**inputs, output_attentions=True)

    # Attention shape: [batch_size, num_heads, seq_len, seq_len]
    attentions = outputs.attentions
    attention_layer = attentions[layer_idx][0]  # Get first batch

    return attention_layer, tokenizer.convert_ids_to_tokens(inputs["input_ids"][0])

# 5. Visualize attention patterns
sample_text = "Artificial intelligence is transforming education"
attention_layer, tokens = get_attention_patterns(sample_text, layer_idx=3)

# Plot attention for one head
fig, axes = plt.subplots(2, 4, figsize=(16, 8))
```

```
fig.suptitle(f"Phi-3-mini Attention Patterns (Layer 3) - '{sample_text}'",
fontsize=14)

for head_idx in range(8):  # Phi-3-mini has 32 heads total, showing first 8
    ax = axes[head_idx // 4, head_idx % 4]
    im = ax.imshow(attention_layer[head_idx].numpy(), cmap='viridis', aspect='auto')
    ax.set_xticks(range(len(tokens)))
    ax.set_yticks(range(len(tokens)))
    ax.set_xticklabels(tokens, rotation=45, fontsize=8)
    ax.set_yticklabels(tokens, fontsize=8)
    ax.set_title(f"Head {head_idx}")
    plt.colorbar(im, ax=ax)

plt.tight_layout()
plt.show()

# Discussion Questions:
# 1. What patterns do you notice in the attention matrices?
# 2. How does tokenization affect the sequence length?
# 3. Based on the architecture specs, why is Phi-3-mini considered "small"?
```

Written Task: Compare Phi-3-mini's architecture with GPT-3. What design choices make Phi-3 efficient? Consider parameter count, layers, and attention heads.

Text Classification with Phi-3-mini (Chapter 4)

Objective: Fine-tune Phi-3-mini for a text classification task using parameter-efficient fine-tuning

python

```
# Coding Exercise - Fine-tuning Phi-3-mini for Sentiment Analysis
# We'll use LoRA (Low-Rank Adaptation) for efficient fine-tuning

# Install: pip install transformers datasets peft torch accelerate bitsandbytes
```

```python
from datasets import load_dataset
from transformers import (
    AutoTokenizer,
    AutoModelForCausalLM,
    TrainingArguments,
    Trainer,
    DataCollatorForLanguageCompletionLM
)
from peft import LoraConfig, get_peft_model, TaskType
import evaluate
import numpy as np
import torch

# 1. Load dataset - using IMDB sentiment analysis
print("Loading dataset...")
dataset = load_dataset("imdb")
print(f"Dataset loaded: {len(dataset['train'])} training samples")

# 2. Load Phi-3-mini tokenizer and model
model_path = "./phi-3-mini-4k-instruct"  # Update with your local path

tokenizer = AutoTokenizer.from_pretrained(model_path, trust_remote_code=True)
# Add padding token if not present
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

model = AutoModelForCausalLM.from_pretrained(
    model_path,
    torch_dtype=torch.float32,  # Use torch.float16 if GPU memory available
    device_map="auto",
    trust_remote_code=True
)

# 3. Prepare dataset for instruction fine-tuning
def format_instruction(example):
    """Format data for instruction fine-tuning"""
    instruction = "Classify the sentiment of this movie review as positive or
negative."
```

```
    # Create prompt template
    prompt = f"""<|user|>
{instruction}

Review: {example['text']}

<|assistant|>
Sentiment:"""

    # Add label to completion
    label = "positive" if example['label'] == 1 else "negative"
    completion = f" {label}"

    # Tokenize
    full_text = prompt + completion
    tokenized = tokenizer(full_text, truncation=True, max_length=512)

    # Create labels (mask prompt tokens with -100)
    prompt_tokens = tokenizer(prompt, truncation=True, max_length=512)["input_ids"]
    completion_tokens = tokenizer(completion, truncation=True, max_length=512,
add_special_tokens=False)["input_ids"]

    labels = [-100] * len(prompt_tokens) + completion_tokens

    return {
        "input_ids": tokenized["input_ids"],
        "attention_mask": tokenized["attention_mask"],
        "labels": labels
    }

# Prepare smaller dataset for faster training
small_train = dataset["train"].shuffle(seed=42).select(range(500))
small_test = dataset["test"].shuffle(seed=42).select(range(100))

print("Formatting dataset...")
train_dataset = small_train.map(format_instruction,
remove_columns=small_train.column_names)
test_dataset = small_test.map(format_instruction,
remove_columns=small_test.column_names)
```

```python
# 4. Configure LoRA for parameter-efficient fine-tuning
lora_config = LoraConfig(
    task_type=TaskType.CAUSAL_LM,
    r=8,  # Rank
    lora_alpha=32,
    lora_dropout=0.1,
    target_modules=["q_proj", "v_proj", "k_proj", "o_proj"],  # Attention modules
    bias="none"
)

# Apply LoRA to model
model = get_peft_model(model, lora_config)
model.print_trainable_parameters()  # Show how many parameters we're training

# 5. Set up training
training_args = TrainingArguments(
    output_dir="./phi3-sentiment",
    num_train_epochs=3,
    per_device_train_batch_size=4,  # Small batch size for memory constraints
    per_device_eval_batch_size=4,
    gradient_accumulation_steps=4,
    warmup_steps=50,
    logging_steps=25,
    eval_strategy="steps",
    eval_steps=100,
    save_strategy="epoch",
    load_best_model_at_end=True,
    metric_for_best_model="eval_loss",
    fp16=torch.cuda.is_available(),  # Use mixed precision if GPU available
    push_to_hub=False,
    report_to="none"
)

# 6. Custom accuracy metric
def compute_metrics(eval_pred):
    predictions, labels = eval_pred

    # Extract predictions (taking the first token after "Sentiment:")
```

```
    # This is simplified - in practice, you'd decode and parse the output
    predictions = np.argmax(predictions, axis=-1)

    # Calculate accuracy
    predictions = predictions.flatten()
    labels = labels.flatten()

    # Filter out padding tokens
    mask = labels != -100
    predictions = predictions[mask]
    labels = labels[mask]

    accuracy = (predictions == labels).mean()
    return {"accuracy": accuracy}

# 7. Create Trainer
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    tokenizer=tokenizer,
    compute_metrics=compute_metrics,
)

# 8. Train the model
print("Starting training...")
trainer.train()

# 9. Test the fine-tuned model
def predict_sentiment(text):
    """Use the fine-tuned model for sentiment prediction"""
    prompt = f"""<|user|>
Classify the sentiment of this movie review as positive or negative.

Review: {text}

<|assistant|>
Sentiment:"""
```

```
    inputs = tokenizer(prompt, return_tensors="pt", truncation=True, max_length=512)

    with torch.no_grad():
        outputs = model.generate(
            **inputs,
            max_new_tokens=10,
            temperature=0.1,
            do_sample=True,
            pad_token_id=tokenizer.pad_token_id
        )

    response = tokenizer.decode(outputs[0], skip_special_tokens=True)
    # Extract just the sentiment part
    sentiment_part = response.split("Sentiment:")[-1].strip()
    return sentiment_part

# Test examples
test_reviews = [
    "This movie was absolutely fantastic! The acting was superb and the plot kept me
engaged from start to finish.",
    "I was disappointed by the poor character development and predictable
storyline.",
    "The cinematography was beautiful but the dialogue felt forced and unnatural."
]

print("\n=== Sentiment Predictions ===")
for review in test_reviews:
    sentiment = predict_sentiment(review[:200])  # Limit length
    print(f"Review: {review[:100]}...")
    print(f"Predicted sentiment: {sentiment}\n")

# Extension Challenges:
# 1. Try different LoRA configurations (rank, alpha, target modules)
# 2. Compare full fine-tuning vs LoRA fine-tuning
# 3. Create a confusion matrix for error analysis
# 4. Implement a more sophisticated prompt template
```

Alternative: Zero-shot Classification with Phi-3-mini (No Fine-tuning)

```python
# Quick demonstration of zero-shot classification with Phi-3-mini
def zero_shot_classification(text, categories):
    """Perform zero-shot classification using Phi-3-mini"""

    prompt = f"""<|user|>
Classify the following text into one of these categories: {', '.join(categories)}.

Text: {text}

Which category does this text belong to? Respond with only the category name.

<|assistant|>
Category:"""

    inputs = tokenizer(prompt, return_tensors="pt", truncation=True, max_length=512)

    with torch.no_grad():
        outputs = model.generate(
            **inputs,
            max_new_tokens=20,
            temperature=0.1,
            do_sample=False,
            pad_token_id=tokenizer.pad_token_id
        )

    response = tokenizer.decode(outputs[0], skip_special_tokens=True)
    return response.split("Category:")[-1].strip()

# Test zero-shot
categories = ["politics", "technology", "sports", "entertainment"]
sample_text = "The new smartphone features groundbreaking AI capabilities and
improved battery life."
result = zero_shot_classification(sample_text, categories)
print(f"Text: {sample_text}")
print(f"Zero-shot classification: {result}")
```

Written Task: Compare the fine-tuning approach vs. zero-shot classification. When would you choose each method? Consider factors like available labeled data, computational resources, and required accuracy.